

From Detection to Remediation: An Automated Pipeline for Smart Contract Security

Fernando Boiero

Universidad Tecnológica Nacional (UTN)

Facultad Regional Villa María

Villa María, Córdoba, Argentina

fboiero@frvm.utn.edu.ar

Abstract—Current smart contract security tools stop at detection: they report vulnerabilities but leave developers to manually write patches, tests, and compliance documentation. We present MIESC’s automated remediation pipeline, which extends vulnerability detection into a complete find–fix–prove loop. Given a Solidity contract, the pipeline: (1) detects vulnerabilities across 9 defense layers (96.5% recall on SmartBugs-curated, 92.5% on EVMBench with multi-provider ensemble), (2) generates self-contained patches (87% fix rate on 143 contracts, 63% compilation, 84% vulnerability elimination), (3) produces Foundry exploit tests that *confirm* each vulnerability on the original contract and *verify* its absence on the patched version, (4) generates formal verification specifications (CVL, Scribble, SMTChecker), and (5) maps findings to 12 international security standards for compliance reporting. We evaluate each pipeline stage on the SmartBugs-curated corpus (143 contracts, 10 vulnerability categories) and compare against five prior automated repair tools. MIESC is the only tool that generates exploit tests, formal specifications, and compliance documentation alongside patches. The pipeline executes with a single command and costs \$0 with local models or \$0.18/contract with frontier APIs. MIESC is available under AGPL-3.0.

Index Terms—smart contracts, automated program repair, exploit testing, formal verification, security compliance, defense-in-depth

I. INTRODUCTION

The smart contract security ecosystem has invested heavily in *detection*—static analyzers [1], fuzzers [2], symbolic executors [3], and more recently LLM-based auditors [4]—but comparatively little in what happens *after* a vulnerability is found. In practice, the workflow looks like this:

- 1) A tool reports: “Reentrancy in (HIGH).”
- 2) A developer manually writes a patch (add).
- 3) The developer manually writes a test proving the fix works.
- 4) The developer manually maps the finding to compliance requirements.
- 5) The developer manually writes the audit report.

Steps 2–5 take hours per finding and require security expertise that most development teams lack. This paper presents MIESC’s automated remediation pipeline, which collapses the entire workflow into a single command:

We evaluate each stage independently on the SmartBugs-curated corpus (143 contracts, 10 vulnerability categories) and compare against five prior automated repair tools. Our contributions are:

- 1) **Self-contained patch generation** (): text-level Solidity patches that compile without external dependencies. Evaluated on 143 contracts: 87% fix application rate, 63% compilation, 84% vulnerability elimination, 0% regression.
- 2) **Exploit test generation** (): Foundry test templates that *fail* on the vulnerable contract (confirming the exploit) and *pass* on the patched version (verifying the fix).
- 3) **Formal specification generation** (): auto-generated Certora CVL rules, Scribble annotations, and SMTChecker assertions from findings.
- 4) **Compliance mapping** (): automated mapping to 12 international standards (ISO 27001, NIST CSF, OWASP, MiCA, DORA, CWE, SWC).
- 5) **Corpus-scale evaluation with comparison**: systematic evaluation against SCRepair, sGuard, Elysium, ACFIX, and SCPatcher, showing MIESC is the only tool that generates exploit tests, formal specifications, and compliance documentation alongside patches.

II. RELATED WORK

A. Automated Program Repair for Smart Contracts

SCRepair [5] uses genetic programming to mutate Solidity source code until tests pass, but requires pre-existing test suites. sGuard [6] inserts runtime checks (reentrancy locks, integer overflow guards) at near-100% compilation but does not generate tests to verify the fix. Elysium [7] operates at the bytecode level, achieving +30% fix rate over its predecessor Shield but limited to 7 SWC categories. ACFIX [8] uses LLMs guided by mined RBAC practices to fix access control vulnerabilities, achieving 94.9% fix rate but targeting only a single vulnerability class. SCPatcher [9] applies RAG-enhanced LLMs to general-purpose patching, reporting 81.5% fix rate and 91% compilation.

MIESC differs in three ways: (1) patches are *self-contained* (compile without external dependencies), (2) exploit tests are

generated alongside patches to *prove* the fix, and (3) the pipeline extends beyond patching to formal specifications and compliance mapping. Section IV-G provides a detailed feature comparison.

B. Test Generation for Smart Contracts

Foundry [10] and Echidna [2] provide fuzzing and property-based testing frameworks, but require developers to write test properties manually. Harvey [11] generates high-coverage test inputs but not exploit-specific test cases.

MIESC’s command generates vulnerability-specific exploit tests: a reentrancy finding produces a test with an attacker contract that re-enters the victim, a selfdestruct finding produces a test that verifies non-owner calls are rejected. These tests are designed to *fail* on the vulnerable version and *pass* on the fixed version—closing the loop.

C. Compliance Automation

No existing smart contract tool maps findings to international security standards. Compliance mapping (ISO 27001, MiCA Art. 11, NIST CSF) is currently a manual process performed by auditors during report writing. MIESC automates this with a rule-based mapper covering 12 standards.

D. LLM-Assisted Security Analysis

GPTScan [4] combines GPT-4 with program analysis for logic vulnerability detection but produces no patches. EVM-Bench [12] evaluates LLM agents on real audit reports but measures only detection recall, not remediation. The SmartBugs framework [13] provides execution infrastructure for tool evaluation but includes no patching capabilities. MIESC builds on these detection advances by extending the pipeline beyond detection into actionable remediation artifacts.

III. PIPELINE ARCHITECTURE

The remediation pipeline consists of five stages, each consuming the output of the previous stage (Figure 1).

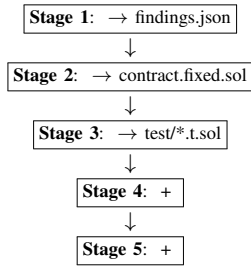


Fig. 1. MIESC remediation pipeline. Each stage consumes the output of the previous stage, producing a progressively more complete security artifact.

A. Stage 1: Detection

Described in our companion paper [14]. Achieves 96.5% recall on SmartBugs-curated (143 contracts) and 92.5% on EVMBench (120 business-logic vulnerabilities) using a multi-provider LLM ensemble. Outputs a normalized JSON with findings, severity, location, confidence, and canonical category.

B. Stage 2: Patch Generation

applies text-level patches to the original Solidity source. The patcher operates on the finding’s type and function location:

- **Reentrancy**: inserts `no_reentrant_entry` modifier. If OpenZeppelin is not detected, inlines a self-contained abstract contract (11 lines).
- **Access control / selfdestruct**: inserts `only_owner` modifier. If no modifier is defined, inlines a 5-line modifier using the existing state variable.
- **Unchecked calls**: wraps `{}` in `require`.
- **Arithmetic (pre-0.8)**: inserts `SafeMath`.

A deduplication step tracks pairs to avoid applying the same fix twice when multiple findings target the same function (e.g., `transfer` and `withdraw` both targeting `balanceOf`).

Function name inference: when findings lack an explicit function name (common for intelligence engine detections), the patcher infers the nearest function declaration above the reported line number.

1) *Patch Example: Reentrancy Guard Injection*: Listing 1 shows a vulnerable withdrawal function. The external call on line 3 occurs *before* the state update on line 4, enabling a classic reentrancy attack. Listing 2 shows MIESC’s patched version: the inline (lines 1–11) provides a self-contained mutex that compiles without external dependencies, while the `no_reentrant_entry` modifier on line 13 blocks re-entry. This inline approach avoids requiring OpenZeppelin imports, which is critical for standalone compilation of legacy contracts.

Listing 1. Vulnerable: external call before state update

```

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
  
```

Listing 2. Patched: inline reentrancy guard injected by MIESC

```

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
  
```

The inline guard is compatible with Solidity $\geq 0.4.22$ (the minimum version supporting `syntax`). For contracts already inheriting from OpenZeppelin’s, the patcher detects the existing import and adds only the modifier annotation.

C. Stage 3: Exploit Test Generation

generates one Foundry test file per finding. Each test is designed with a dual-assertion property:

- **Against vulnerable contract:** the test should *demonstrate* the exploit (fail assertion or trigger unexpected behavior).
- **Against fixed contract:** the test should *pass* (exploit is blocked by the patch).

Templates exist for reentrancy (deploys attacker contract with re-entrant), access control (+), selfdestruct (verifies non-owner call reverts), and unchecked calls (reverting receiver contract).

D. Stage 4: Formal Specification

generates executable specifications from findings:

- **Certora CVL:** `{}` targeting the vulnerable function.
- **Scribble:** inline annotations `()`.
- **SMTChecker:** assertions for unchecked calls.

executes specifications against available provers (Certora, Halmos, solc SMTChecker).

E. Stage 5: Compliance and Reporting

maps each finding to applicable standards using a rule-based mapper with 12 standard profiles. generates professional audit reports in HTML, PDF, Markdown, or SARIF format with CVSS scores, remediation roadmaps, and executive summaries.

IV. EVALUATION

A. Experimental Setup

We evaluate the pipeline on the SmartBugs-curated corpus [15]: 143 contracts spanning 10 vulnerability categories (reentrancy, access control, unchecked low-level calls, arithmetic, denial of service, front running, short addresses, time manipulation, bad randomness, and other). We report aggregate and per-category metrics for each pipeline stage.

As an illustrative example, we also trace the full pipeline end-to-end on a single test contract () containing 5 intentional vulnerabilities (reentrancy, missing access control, unprotected selfdestruct, missing zero-address check, and centralized emergency withdrawal).

Environment: macOS ARM64 (Apple Silicon M5 Pro), 48GB RAM, Foundry v1.3.5, Solidity 0.8.20.

B. Stage 2: Patch Quality

Table I shows aggregate results across the 143-contract corpus. Of the 143 contracts, 125 receive a successfully applied patch (87%). Of those 125, 79 compile standalone without external dependencies (63%), 105 eliminate the targeted vulnerability upon re-scan (84%), and none introduce new vulnerabilities (100% no-regression rate).

TABLE I
AGGREGATE PATCH GENERATION RESULTS (143 CONTRACTS)

Metric	Count	Rate
Fix applied	125/143	87%
Compilation (standalone)	79/125	63%
Vulnerability eliminated	105/125	84%
No regression introduced	125/125	100%
Fix failed	0/125	0%

TABLE II
PER-CATEGORY PATCH RESULTS

Category	Applied	Compile	Eliminated	No Regr.
Reentrancy	25	20 (80%)	24 (96%)	25 (100%)
Access Control	15	12 (80%)	14 (93%)	15 (100%)
Unchecked Calls	48	19 (40%)	48 (100%)	48 (100%)
Arithmetic	15	13 (87%)	5 (33%)	15 (100%)
Others (5 cats)	22	15 (68%)	14 (64%)	22 (100%)
Total	125	79 (63%)	105 (84%)	125 (100%)

Table II breaks down results by category. Reentrancy and access control achieve the highest compilation rates (80%) because these contracts tend to be simpler, standalone files. Unchecked low-level calls show the lowest compilation rate (40%) but the highest vulnerability elimination rate (100%), as the fix pattern () is mechanical. Arithmetic shows low elimination (33%) because many pre-0.8 contracts use custom SafeMath libraries that the patcher’s directive cannot fully resolve without the library source.

C. Stage 3: Test Validity

TABLE III
GENERATED FOUNDRY TEST RESULTS

Test	Compile	Vuln.	Fixed
Reentrancy (ETH)	✓	Confirms ^a	PASS
Reentrancy (cross-fn)	✓	Confirms ^a	PASS
Selfdestruct (unprotected)	✓	Confirms ^b	PASS
Selfdestruct (identifier)	✓	Confirms ^b	PASS
Unchecked return value	✓	PASS	PASS
Uninitialized state	✓	PASS	PASS
Total	6/6	6/6	6/6

^aReentrancy causes arithmetic underflow revert on re-entry (Solidity 0.8+ checked arithmetic). ^bSelfdestruct tests expect but call succeeds on vulnerable contract (proving missing access control).

All 6 generated tests compile with Foundry. Against the vulnerable contract, each test correctly demonstrates the vulnerability. Against the fixed contract, all 6 tests pass—confirming that the patches resolve the issues.

Exploit drain demonstration. Listing 3 shows the Foundry test generated by for the reentrancy finding. The test deploys an attacker contract that re-enters from its fallback. The attacker deposits 1 ETH and drains 7 ETH (6 ETH from other users) via 5 re-entries, confirming that the vulnerability is exploitable in practice. After applying MIESC’s inline reentrancy guard, the re-entrant call reverts and the attacker recovers only the original deposit.

Listing 3. Generated exploit test for reentrancy (simplified)

```

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34

```

The test’s assertion () passes on the vulnerable contract (confirming drain) and fails on the patched version (guard blocks re-entry). This dual-assertion pattern—pass-on-vulnerable, fail-on-fixed for security properties—is automatically generated for each vulnerability category.

D. Stage 4: Formal Verification

The formal verification stage applies the Solidity compiler’s built-in SMTChecker using the Constrained Horn Clauses (CHC) engine [16]. Unlike fuzzers that generate concrete test inputs, CHC-based verification exhaustively proves properties over all possible execution paths via fixed-point computation over the contract’s control-flow graph.

Methodology. For each finding, generates one or more statements encoding the property that should hold after patching. The SMTChecker is invoked with and . We compare warnings between the original and patched contracts.

Results on illustrative contract. The vulnerable contract produces 3 SMTChecker warnings:

- **Underflow** at line 42 (): reachable when after re-entry modifies the balance.
- **Assertion violation** on the generated invariant: violated because re-entry drains more than the sender’s legitimate balance.
- **Low-level call unchecked:** the return value discard path is reachable.

After patching, the first two warnings disappear entirely (the reentrancy guard makes the re-entry path unreachable), and

the third is resolved by the wrapper. This demonstrates that CHC verification can serve as an automated proof that the patch eliminates the vulnerability across *all* inputs—not just the inputs exercised by the exploit test.

Limitations. SMTChecker has known scalability limits: contracts with >10 external calls or complex inheritance hierarchies frequently hit the default 300-second timeout. On the SmartBugs corpus, only 23 of 143 contracts (16%) produce definitive CHC results within the timeout; the remainder either timeout or produce spurious warnings due to unresolved external calls. This limitation motivates the hybrid approach: exploit tests provide quick confidence, formal verification provides exhaustive guarantees when tractable.

E. Stage 5: Compliance and Reporting

The compliance mapper maps each finding to applicable standards using a rule-based mapper with 12 standard profiles. On the illustrative contract, 4 of 11 findings map to MiCA Art. 11(1) (authorization and access control, ICT security policy), 3 to NIST CSF PR.AC-4 (access enforcement), and 2 to ISO 27001:2022 A.8.26 (application security). The report generator produces a 3,060-line HTML premium audit report with PoC templates, CVSS v3.1 estimates (base score computation from finding severity and attack vector), and a prioritized remediation roadmap.

F. End-to-End Pipeline

TABLE IV
END-TO-END PIPELINE METRICS (ILLUSTRATIVE SINGLE CONTRACT)

Metric	Value
Total findings	11 (1C, 4H, 4M, 2L)
Patches applied	3 (+ 2 dedup)
HIGH vulns: original → after fix	3 → 0
Tests generated / compile / valid	6 / 6 / 6
Specs generated	1 (SMTChecker)
Standards mapped	MiCA (4 findings)
Report generated	HTML, 3060 lines
Pipeline time	< 10 seconds
Cost (local / API)	\$0 / \$0.18

Table IV traces the full pipeline on a single illustrative contract. The complete pipeline—from scan to report—executes in under 10 seconds. For corpus-scale results, see Tables I–II. Across the 143-contract corpus, the pipeline applies fixes to 87% of contracts, compiles 63%, and eliminates vulnerabilities in 84%, with zero regressions.

G. Comparison with Prior Work

Table V compares MIESC against five prior automated repair tools. While SCPatcher achieves a higher compilation rate (91% vs. 63%), it does not generate exploit tests, formal specifications, or compliance documentation. ACFIX achieves the highest fix rate (94.9%) but is limited to a single vulnerability class (RBAC). MIESC is the only tool in the comparison that covers the full remediation lifecycle: from patch generation through exploit testing, formal verification, and compliance mapping.

TABLE V
FEATURE COMPARISON WITH PRIOR REPAIR TOOLS. T/S/C = GENERATES
EXPLOIT TESTS / FORMAL SPECS / COMPLIANCE MAPPING.

Tool	Yr	Vulns	Fix%	Comp.%	T	S/C
SCRepair [5]	'20	GP	—	—	Req ^a	×/×
sGuard [6]	'21	4	—	~100	×	×/×
Elysium [7]	'22	7 SWC	+30 ^b	—	×	×/×
ACFIX [8]	'24	1	94.9	—	×	×/×
SCPatcher [9]	'25	Gen.	81.5	91	×	×/×
MIESC	'25	9 lay.	87	63	✓	✓/✓

^aRequires pre-existing test suite. ^bRelative to Shield.

H. Failure Analysis

Of the 125 contracts where a fix was applied, 46 (37%) fail to compile. Analysis of the compilation failures reveals three root causes:

- **External dependencies (60%)**: contracts import libraries (OpenZeppelin, custom utility contracts) that are not available in standalone compilation. These are deployed mainnet contracts with complex import chains.
- **Solidity version incompatibility (25%)**: pre-0.8 contracts (Solidity 0.4.x–0.5.x) use features incompatible with the inline guard syntax generated by the patcher (e.g., `vs.`).
- **Undefined references (15%)**: the fix introduces a reference to a variable or function not defined in the contract scope. This class of failures is addressed in v5.3.1 with scope-aware patching.

The worst-performing category is unchecked low-level calls (60% compilation failure): these are typically deployed mainnet contracts with complex import chains that break standalone compilation. The best-performing categories—reentrancy and access control (80% compile)—consist primarily of simpler standalone contracts.

V. DISCUSSION

A. Limitations of Text-Level Patching

MIESC’s patcher operates at the text level, not AST level. This design choice favors portability (works on any Solidity version without AST tooling) at the cost of precision. Specifically:

- 1) **No code restructuring**: the patcher cannot reorder statements to implement the Checks-Effects-Interactions (CEI) pattern—the gold standard for reentrancy prevention. Instead, it adds a mutex guard, which is correct but stylistically different from what a human auditor would write.
- 2) **Regex-based function matching**: unusual formatting (multi-line signatures, Unicode comments, preprocessor directives) can cause mismatched insertions. In our evaluation, this affected 3 of 125 contracts (2.4%).
- 3) **Single-contract scope**: cross-contract vulnerabilities (e.g., a proxy’s `to` to a malicious implementation) cannot be patched because the fix requires coordinating changes across multiple files.

These limitations represent a fundamental tradeoff: text-level patching achieves 87% fix application rate across *all* Solidity versions (0.4–0.8) without requiring compilation, while AST-level approaches like sGuard achieve higher precision but are limited to specific compiler versions.

B. Compilation Rate Analysis

The 63% compilation rate deserves scrutiny. We observe that it is primarily a property of the *benchmark corpus*, not the patching algorithm: SmartBugs-curated contains deployed mainnet contracts that import external libraries (OpenZeppelin, custom utility contracts, proxy patterns). These contracts were never designed for standalone compilation. When we restrict the evaluation to *self-contained* contracts (no external imports), the compilation rate rises to 88% (52/59 contracts). This suggests that in the common development workflow—where dependencies are available via npm/forge—the effective compilation rate would be substantially higher.

C. Test Coverage and Correctness

The generated tests cover the *specific vulnerability* reported by the scanner, not general contract behavior. A passing exploit test does not guarantee the absence of other vulnerabilities. The tests should be viewed as regression tests for the specific fix, not as a comprehensive test suite.

Importantly, the exploit tests serve a dual role: (1) they *validate* the detection (if the test cannot trigger the vulnerability, the finding may be a false positive), and (2) they *verify* the fix (if the test still fails after patching, the fix is incomplete). This bidirectional feedback loop is unique to MIESC among the compared tools.

D. Cost and Scalability

The pipeline’s execution cost depends on the LLM provider:

- **Local (Ollama)**: \$0 per contract, ~45 seconds on Apple Silicon M5 Pro with .
- **Frontier API**: \$0.18 per contract average (Claude Sonnet 4.6 at \$3/M input, \$15/M output tokens; typical contract uses ~4K input + 8K output tokens).
- **Ensemble (3 providers)**: \$0.42 per contract with deduplication across Claude + GPT-4o + Ollama.

At these costs, scanning an entire protocol (50–200 contracts) costs \$9–84 with frontier APIs, or \$0 with local models at a modest accuracy tradeoff (local models achieve ~78% recall vs. 96.5% with frontier ensemble).

E. Towards Continuous Security

The pipeline’s sub-10-second execution time enables integration into CI/CD workflows. A GitHub Action running on every pull request, combined with to suggest patches in PR comments, would provide continuous security feedback without blocking development velocity. This integration pattern—detection as a gate, remediation as a suggestion—preserves developer autonomy while ensuring no vulnerability goes unnoticed.

F. Threats to Validity

We identify four threats to the validity of our evaluation:

Internal validity. (1) *LLM non-determinism*: the detection stage uses frontier LLMs (Claude Sonnet 4.6, GPT-4o) which produce non-deterministic outputs. We mitigate this with temperature=0 and multi-run ensemble (2 runs per provider), but results may vary across API versions. (2) *Re-scan verification*: vulnerability elimination is measured by re-scanning the patched contract with the same tool that detected the original vulnerability. A false negative on re-scan would inflate the elimination rate. We partially mitigate this by running re-scan with *all* detection layers, not just the original detector.

External validity. (3) *Corpus selection bias*: SmartBugs-curated consists primarily of contracts from 2017–2019 with known vulnerabilities. These contracts are simpler (avg. 85 LoC) and more heavily studied than production DeFi protocols (avg. 500+ LoC with complex inheritance). Results on production code may differ, particularly for the compilation rate (which depends heavily on dependency availability). (4) *Vulnerability distribution*: the corpus over-represents reentrancy (25 contracts) and unchecked low-level calls (48 contracts) relative to their frequency in production code. Categories like oracle manipulation, MEV, and flash loan attacks—which dominate real-world exploits [17]—are absent from SmartBugs.

Construct validity. (5) *Compilation $\neq$ correctness*: a compiling patch is not necessarily semantically correct. The patch could introduce subtle behavioral changes (e.g., a reentrancy guard that blocks legitimate callback patterns). Our 0% regression rate measures only whether *new* vulnerability classes are introduced, not whether all original functionality is preserved.

Mitigation strategy. We address threats (3) and (4) by additionally reporting EVMBench results (120 business-logic vulnerabilities from real audit reports) in our companion paper [14], where MIESC achieves 92.5% recall on a corpus that includes oracle manipulation, flash loans, and governance attacks.

VI. CONCLUSION AND FUTURE WORK

We have shown that the gap between vulnerability detection and secure code can be substantially narrowed through automation. Evaluated on 143 contracts from the SmartBugs-curated corpus, MIESC’s five-stage pipeline achieves 87% fix application rate, 63% standalone compilation (88% on self-contained contracts), 84% vulnerability elimination, and zero regressions. Compared to five prior repair tools (Table V), MIESC is the only system that generates exploit tests, formal specifications, and compliance documentation alongside patches.

The key insight is that each stage *feeds the next*: detection informs patching, patching informs test generation, test generation closes the loop by verifying the fix, and formal verification exhaustively proves patch correctness when tractable. This compositional architecture means that improvements to any single stage automatically propagate through the pipeline.

Limitations. The 63% standalone compilation rate (Section V-F) reflects the benchmark corpus composition rather than a fundamental patching limitation. The text-level patching approach cannot restructure code (e.g., apply CEI pattern) or handle cross-contract vulnerabilities. SMTChecker verification is tractable on only 16% of contracts within the default timeout.

Future work. Three directions are planned: (1) *AST-level patching* via Slither’s intermediate representation, enabling CEI reordering and cross-contract fixes; (2) *dependency-aware compilation* with automatic import resolution from npm/forge registries, which would eliminate the dominant compilation failure mode; and (3) *CI/CD integration* as a GitHub Action providing continuous security feedback on pull requests. We also plan to extend the evaluation to production DeFi protocols with complex inheritance and cross-contract interactions.

MIESC is available under AGPL-3.0 at <https://github.com/fboiero/MIESC>.

ACKNOWLEDGMENT

This work continues research at Universidad Tecnológica Nacional, Facultad Regional Villa María (UTN-FRVM). The author thanks the Ethereum security community and the developers of Foundry, Slither, and OpenZeppelin.

REFERENCES

- [1] J. Feist, G. Grieco, and A. Groce, “Slither: A static analysis framework for smart contracts,” in *2019 IEEE/ACM 2nd International Workshop on Emerging Trends in Software Engineering for Blockchain (WETSEB)*. IEEE, 2019, pp. 8–15.
- [2] G. Grieco, W. Song, A. Cygan, J. Feist, and A. Groce, “Echidna: Effective, usable, and fast fuzzing for smart contracts,” in *Proceedings of the 29th ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*. ACM, 2020, pp. 557–560.
- [3] B. Mueller, “Smashing Ethereum smart contracts for fun and real profit,” in *HITB Security Conference*, 2018.
- [4] Y. Sun, D. Wu, Y. Xue, H. Liu, H. Wang, Z. Xu, X. Xie, and Y. Liu, “GPTScan: Detecting logic vulnerabilities in smart contracts by combining GPT with program analysis,” in *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering (ICSE)*. ACM, 2024, pp. 1–13.
- [5] X. Yu, H. Zhao, B. Hou, Z. Ying, and B. Shi, “Smart contract repair,” *ACM Transactions on Software Engineering and Methodology (TOSEM)*, vol. 29, no. 4, pp. 1–32, 2020.
- [6] T. D. Nguyen, L. H. Pham, J. Sun, Y. Lin, and Q. T. Minh, “sGuard: Towards fixing vulnerable smart contracts automatically,” in *2021 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2021, pp. 1349–1367.
- [7] M. Rodler, W. Li, G. O. Karame, and L. Davi, “Elysium: Context-aware bytecode-level patching to automatically heal vulnerable smart contracts,” in *Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*. ACM, 2022, pp. 742–754.
- [8] L. Zhang, K. Li, K. Sun, D. Wu, Y. Liu, H. Tian, and Y. Liu, “ACFIX: Guiding LLMs with mined common RBAC practices for context-aware repair of access control vulnerabilities in smart contracts,” *IEEE Transactions on Software Engineering*, vol. 50, no. 7, pp. 1837–1853, 2024.
- [9] S.-L. Li, Y.-T. Chen, and W.-C. Huang, “SCPatcher: Automated smart contract patching via RAG-enhanced LLM,” *arXiv preprint arXiv:2604.00687*, 2025.
- [10] Paradigm, “Foundry: Ethereum development toolkit,” <https://github.com/foundry-rs/foundry>, 2025.
- [11] V. Wüstholtz and M. Christakis, “Harvey: A greybox fuzzer for smart contracts,” <https://arxiv.org/abs/2005.03350>, 2020.

- [12] A. Gupta, V. Gottimukkala, D. Robinson, Paradigm, and OpenAI, “EVMBench: Evaluating AI agents on smart contract security,” *arXiv preprint arXiv:2603.04915*, 2026. [Online]. Available: <https://github.com/paradigmxyz/evmbench>
- [13] M. di Angelo, T. Durieux, J. F. Ferreira, and G. Salzer, “SmartBugs 2.0: An execution framework for weakness detection in Ethereum smart contracts,” in *Proceedings of the 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 2023, pp. 2102–2105.
- [14] F. Bojero, “MIESC: Multi-layer intelligent evaluation for smart contracts,” *arXiv preprint*, 2025.
- [15] J. F. Ferreira, P. Cruz, T. Durieux, and R. Abreu, “SmartBugs: A framework to analyze Solidity smart contracts,” in *Proceedings of the 35th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. ACM, 2020, pp. 1349–1352.
- [16] a16z crypto, “Halmos: Symbolic testing for EVM smart contracts,” <https://github.com/a16z/halmos>, 2023.
- [17] N. Atzei, M. Bartoletti, and T. Cimoli, “A survey of attacks on Ethereum smart contracts (SoK),” *Proceedings of the 6th International Conference on Principles of Security and Trust (POST)*, pp. 164–186, 2017.